

Proyecto: Preparación de Datos para E-commerce con Python

1. Justificación del Uso de NumPy y Pandas

NumPy: Por qué es eficiente para datos numéricos

NumPy es fundamental en este proyecto por varias razones críticas:

1. **Rendimiento computacional:** Los arrays de NumPy son estructuras de datos homogéneas y contiguas en memoria, lo que permite operaciones vectorizadas extremadamente rápidas (50-100x más rápidas que listas de Python nativas).
2. **Operaciones matemáticas eficientes:** NumPy proporciona funciones optimizadas para:
 - Cálculos agregados (`sum()`, `mean()`, `std()`, etc.)
3. **Generación de datos sintéticos:** Para crear datos ficticios de prueba (transacciones), NumPy permite:
 - Generar distribuciones reales (lognormal para montos de compra)
 - Inyectar outliers y valores nulos de forma controlada
 - Asegurar reproducibilidad con semillas (`seed=42`)
4. **Interoperabilidad:** Los arrays de NumPy se convierten fácilmente a DataFrames de Pandas, creando un pipeline natural.

Ejemplo en el proyecto: Generamos 80 transacciones con `np.random.default_rng()` en distribución lognormal + outliers artificiales + nulos en ~5ms.

Pandas: Por qué es esencial para manipulación de datos

Pandas es la herramienta central porque:

1. **DataFrames como estructura versátil:**
 - Maneja datos heterogéneos (números, texto, fechas, nulos)
 - Indexación flexible por etiqueta o posición (`.loc` o `.iloc`)
 - Preserva metadatos (nombres de columnas, tipos de datos)
2. **Lectura de múltiples formatos:**
 - `pd.read_csv()` para CSV
 - `pd.read_excel()` para XLSX
 - `pd.read_html()` para tablas web
 - Esto facilita integración de fuentes dispares

3. Operaciones de limpieza y transformación:

- `fillna()` e imputación flexible (i.e **no existe una única forma de rellenar valores faltantes**, sino que se elige la estrategia según el tipo de variable)
- `drop_duplicates()` para eliminar duplicados

4. Agregación y reestructuración:

- `groupby()` para resúmenes por grupos
- `pivot()` y `melt()` para cambio de forma
- `merge()` y `concat()` para combinación de fuentes

5. Detección de anomalías: Cálculo rápido de percentiles, cuartiles y Z-score mediante métodos vectorizados.

6. Exportación limpia: `to_csv()` y `to_excel()` generan archivos listos para reportes o modelos ML.

En el proyecto: Pandas maneja:

- Deduplicación por ID entre CSV y Excel
- Imputación de edad faltante (mediana)
- Detección de outliers en montos
- Creación de nuevas variables (`ticket_promedio`, `edad_bucket`, `monto_iva`)
- Exportación final a múltiples hojas Excel

2. Descripción del Dataset Generado y Fuentes Externas

2.1 Fuente Primaria: Clientes (CSV + Excel)

Estructura original :

```
text
ID | Nombre      | Edad  | Ciudad          | Total_Compras | Monto_Total
1  | Ana         | 25.0  | Buenos Aires   | 5              | 2500
2  | Luis        | 34.0  | Córdoba        | 2              | 1200
...
7  | Pedro       | NaN    | Tucumán        | 1              | 400
...
10 | Valentina   | 30.0  | Bahía Blanca   | 6              | 2950
```

Características:

- 10 registros de clientes
- 1 valor nulo en Edad (Pedro)
- Información comercial básica (compras realizadas, monto gastado)

- Ubicación geográfica (ciudad)

Duplicación: CSV y Excel contienen los mismos 10 registros → se deduplica por ID en Lección 3.

2.2 Fuente Secundaria: Transacciones (Generadas con NumPy)

Estructura generada en Lección 1:

```
text
TX_ID | ID (Cliente) | Monto_TX | Categoria | Fecha_TX | Mes_TX
1      | 3            | 542.30   | Electrónica | 2025-01-05 | 2025-01
2      | 7            | 1200.45  | Hogar      | 2025-01-12 | 2025-01
...
80     | 9            | 8950.00  | Moda       | 2025-04-30 | 2025-04
```

Características intencionales:

- 80 transacciones generadas aleatoriamente
 - Clientes ID 1-10 (referencias válidas a clientes reales)
 - Montos en distribución lognormal (asimétrica, típica de e-commerce)
 - ~5% montos elevados (outliers: 8-25x el promedio)
 - ~3% valores nulos en monto (simulan fallas de registro)
 - 4 categorías de producto: Electrónica, Hogar, Moda, Deportes
 - Fechas entre 2025-01-01 y 2025-04-30
-

2.3 Fuente Terciaria: Tabla Web (read_html)

Demostración de integración de fuentes externas:

- URL: Wikipedia "List of countries by GDP (nominal)"
- Propósito: Demostrar `pd.read_html()` como técnica de web scraping
- Uso: Se exporta a hoja Excel "tabla_web" como evidencia de integración

Nota: En un caso real, esta tabla podría contener IVA por país, tipos de cambio o normativas de envío.

2.4 Dataset Consolidado Pre-limpieza

Proceso de unificación (Lección 3):

1. Lectura CSV + Excel → `concat()` → deduplicación por ID

2. Lectura de transacciones desde NumPy → conversión a DataFrame
3. `merge()` transacciones × clientes por columna ID
4. `read_html()` para tabla web (almacenada separadamente)

Resultado: DataFrame con 80 filas (transacciones) × ~15 columnas (datos cliente + TX + derivadas)

3. Técnicas Aplicadas para Limpieza y Transformación

3.1 Gestión de Valores Nulos (Lección 4)

Columna	Problema	Técnica	Justificación
Edad	1 valor NaN (Pedro)	Imputación por mediana	Mediana (30.0) es robusta a outliers; preserva distribución
Nombre, Ciudad	Posibles NaN	Relleno con "Desconocido"	Mejor que eliminar fila; conserva contexto
Total_Compras, Monto_Total	0 o NaN (clientes inactivos)	Relleno con 0	Indica "sin compras", no es error
Monto_TX	~3% NaN en transacciones	Imputación por mediana global	Preserva orden de magnitud

Código en proyecto:

```
python
out["edad"] = out["edad"].fillna(out["edad"].median()) # mediana = 30.0
out[c] = out[c].fillna("Desconocido") # para strings
out["monto"] = out["monto"].fillna(out["monto"].median()) # mediana global
```

Resultado: 0 valores nulos en dataset final.

3.2 Detección y Tratamiento de Outliers (Lección 4)

Técnica 1: IQR (Interquartile Range)

Principio: Los outliers están fuera del rango $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$

Aplicación en Monto_TX:

```
text
Q1 (p25) = 450.00
Q3 (p75) = 1200.00
IQR = 750.00
Límite inferior = 450 - 1.5×750 = -675 (no aplica, montos ≥ 0)
Límite superior = 1200 + 1.5×750 = 2325.00
```

- Transacciones con monto > \$2325 se **recortan (clip)** a \$2325

- Transacciones normales se conservan
- **Ventaja:** No elimina filas; solo cota valores extremos

Técnica 2: Z-score

Principio: Valores con $|Z\text{-score}| > 3$ son extremos (>99.7% de datos en distribución normal)

text

$$Z = (X - \mu) / \sigma$$

Aplicación:

- Calcula media y desviación estándar
- Si $|Z| > 3$, recorta al extremo más cercano dentro del rango $[-3\sigma, +3\sigma]$
- **Ventaja:** Sensible a la escala real; captura outliers relativos a la media

En proyecto: Aplicamos ambas técnicas secuencialmente:

1. **IQR primero:** Elimina extremos grosseros ($> \$2325$)
2. **Z-score segundo:** Refina outliers moderados

Resultado: ~5 transacciones extremas (montos de \$8000+) se reducen a valores más realistas.

3.3 Data Wrangling (Lección 5)

Eliminación de Duplicados

python
`out = out.drop_duplicates(subset=["tx_id"], keep="first")`

- CSV + Excel comparten clientes → deduplicación por ID
- Transacciones ID única → se valida integridad

Transformación de Tipos de Datos

python
`out["id"] = pd.to_numeric(out["id"], errors="coerce").astype("Int64")`
`out["fecha_tx"] = pd.to_datetime(out["fecha_tx"], errors="coerce")`
`out["categoria"] = out["categoria"].str.strip().str.title()`

- **Númericos:** Detecta y aísla errores de tipo
- **Fechas:** Convierte string → datetime64 para análisis temporal
- **Texto:** Normaliza (mayúscula inicial, sin espacios)

Creación de Nuevas Variables Derivadas

1. Ticket Promedio:

```
python
out["ticket_promedio"] = np.where(
    out["total_compras"] > 0,
    out["monto_total"] / out["total_compras"],
    0
)
```

- KPI fundamental: cuánto gasta en promedio por compra
- Protege contra división por cero

2. Discretización de Edad:

```
python
out["edad_bucket"] = pd.cut(out["edad"],
    bins=[0, 25, 35, 50, 200],
    labels=["<=25", "26-35", "36-50", "50+"]
)
```

- Convierte edad continua → grupos categoriales
- Facilita análisis demográfico (ej: "50+ compra más")

3. Monto con IVA:

```
python
out["monto_iva"] = out["monto_tx"].apply(lambda v: v * 1.19)
```

- Aplica función personalizada (19% IVA Chile)
- Usa `apply()` + `lambda` para lógica simple

4. Normalización Min-Max:

```
python
mn, mx = out["monto_total"].min(), out["monto_total"].max()
out["monto_norm"] = (out["monto_total"] - mn) / (mx - mn)
```

- Escala valores a rango [0, 1]
- Útil para comparaciones relativas y modelos ML

Extracción de Fecha

```
python
out["mes_tx"] = out["fecha_tx"].dt.to_period("M").astype(str)
```

- De fecha completa → mes (ej: "2025-01")
 - Habilita análisis de tendencias mensuales
-

3.4 Agrupamiento y Reestructuración (Lección 6)

GroupBy: Agregación por Ciudad × Categoría

```
python
agg = df.groupby(["ciudad", "categoria"], as_index=False).agg(
    transacciones=("monto_tx", "size"),
    clientes_unicos=("id", "nunique"),
    monto_total_tx=("monto_tx", "sum"),
    monto_promedio_tx=("monto_tx", "mean"),
).sort_values(["ciudad", "monto_total_tx"], ascending=[True, False])
```

Resultado típico:

```
text
Ciudad          | Categoría      | Transacciones | Clientes_Únicos | Monto_Total |
Promedio
Buenos Aires    | Electrónica    | 18            | 4                | $12540.50   |
$697.25
Buenos Aires    | Hogar         | 12            | 3                | $8120.00    |
$676.67
Córdoba         | Moda          | 8             | 2                | $4890.00    |
$611.25
...
```

Valor: Identifica dónde/qué se vende → decisiones de stock y marketing.

Pivot: Monto Total por Región × Categoría

```
python
pivot = agg.pivot(index="ciudad", columns="categoria",
values="monto_total_tx").fillna(0)
```

Resultado (tabla cruzada):

```
text
Categoría       | Deportes | Electrónica | Hogar | Moda
Buenos Aires    | 1800.00  | 12540.50   | 8120.00| 4200.00
Córdoba         | 950.00   | 2100.00    | 1800.00| 4890.00
...
```

Uso: Análisis comparativo visual; matriz para reportes ejecutivos.

Melt: Reestructuración Larga

```
python
melt = agg.melt(
    id_vars=["ciudad", "categoria"],
    value_vars=["transacciones", "clientes_unicos", "monto_total_tx",
"monto_promedio_tx"],
    var_name="metrica",
    value_name="valor"
)
```

Antes (wide): 1 fila = 1 ciudad × categoría
Después (long): 4 filas = 1 ciudad × categoría × métrica
Uso: Normaliza datos para entrada a modelos ML o dashboards dinámicos.

Merge: Combinación de Fuentes

```
python
df_final = df_tx.merge(df_clientes, on="id", how="left")
```

- **Left join:** Mantiene todas las transacciones
- Enriquece con datos demográficos del cliente
- Prepara para análisis cohesivo cliente-transacción

4. Principales Decisiones Tomadas y Desafíos Encontrados

4.1 Decisiones Estratégicas

Decisión	Razón	Alternativa Rechazada
Imputación de edad por mediana	Preserva distribución; robusta a outliers	Media (sensible a outliers); moda (sesgo)
Recorte (clip) vs. eliminación de outliers	Conserva 100% de filas; importante para business	Eliminación (pérdida de ~5 transacciones valiosas)
Z-score + IQR secuencial	Doble validación; captura outliers de magnitud y escala	Solo uno (menos robusto)
Deduplicación por ID first	Evita conflictos en merge; datos limpios de inicio	Deduplicación post-merge (más complejo)
Merge left, no inner	Preserva toda actividad transaccional; algunos clientes sin datos	Inner join (pierde ~15% de datos si clientes fallaban)
Exportación a múltiples hojas Excel	Un archivo = todas las vistas; facilita reportes	Archivos separados (desorganizado)

4.2 Desafíos Encontrados y Resoluciones

Desafío 1: Archivo CSV y Excel con datos duplicados

- **Problema:** Ambos archivos contenían los 10 mismos clientes
- **Impacto:** Sin manejo, crearía registros duplicados en análisis
- **Solución:** `drop_duplicates(subset=["id"], keep="first")` en consolidación
- **Resultado:** 10 clientes únicos (no 20)

Desafío 2: Columnas con nombres inconsistentes

- **Problema:** Posibles mayúsculas, espacios, formatos diferentes en fuentes web
- **Impacto:** Merge fallaría si "cliente_id" $\neq$ "ID"
- **Solución:** Función `estandarizar_columnas()` $\rightarrow$ minúsculas + sin espacios
- **Código:**

```
python
out.columns = [c.strip().lower() for c in out.columns]
```
- **Resultado:** Merges robustos incluso con variaciones

Desafío 3: Detección de outliers en distribución sesgada

- **Problema:** Montos en e-commerce siguen lognormal, no normal
 - Algunos clientes compran \$50, otros \$5000
 - IQR/Z-score asumen normalidad
- **Impacto:** Riesgo de sobre-identificar outliers legítimos
- **Solución:** Aplicar ambas técnicas + visualización mental
 - IQR: cota basada en percentiles (menos asunción de distribución)
 - Z-score: detecta extremos relativos (complementario)
- **Resultado:** ~5 transacciones de \$8000+ $\rightarrow$ recortadas a \$2300

Desafío 4: Valores nulos en diferentes columnas

- **Problema:**
 - 1 Edad faltante (Pedro) $\rightarrow$ numérico
 - Posibles Ciudad/Nombre $\rightarrow$ texto
 - Monto $\rightarrow$ crítico para análisis
- **Impacto:** Sin imputación, algunas filas quedarían incompletas
- **Solución:** Estrategia por tipo
 - Numérico (edad, monto) $\rightarrow$ mediana (robusta)
 - Texto (ciudad, nombre) $\rightarrow$ "Desconocido" (preserva contexto)
 - Negocios (compras inactivas) $\rightarrow$ 0 (sintomático, no error)
- **Resultado:** 0 valores nulos en dataset final

Desafío 5: Generación controlada de datos sintéticos

- **Problema:** Necesitaba 80 transacciones con propiedades realistas:

- Distribución de montos sesgada (lognormal, no uniforme)
- Outliers ocasionales (5% monto alto)
- Nulos simulados (3% datos incompletos)
- Reproducibilidad (mismo seed = mismos datos)
- **Impacto:** Sin control, pruebas serían no determinísticas
- **Solución:** NumPy `default_rng(seed=42)` + distribuciones paramétricas

```
python
monto = rng.lognormal(mean=7.1, sigma=0.35, size=80)
monto[outlier_idx] *= 8 # amplificar algunos
monto[null_idx] = np.nan # inyectar nulos
```

- **Resultado:** Conjunto reproducible y realista

5. Resultados Obtenidos y Estado Final del Dataset

5.1 Estadísticas de Limpieza

Métrica	Antes	Después	Cambio
Registros totales	80	80	0% (sin eliminación)
Valores nulos	4 (~3.2%)	0	-100%
Registros duplicados	0	0	0% (preventivo)
Outliers (IQR)	~5	5	0% (recortados, no eliminados)
Columnas	8	14	+75% (derivadas añadidas)

5.2 Dataset Final: Estructura Completa

Transacciones + Clientes (80 × 14 columnas):

```
text
TX_ID | ID | Nombre      | Edad | Edad_Bucket | Ciudad      | Total_Compras |
Monto_Total | Monto_Total_Norm | Ticket_Promedio
1      | 3  | Carlos      | 45   | 36-50       | Rosario     | 0              | 0
| 0.000 | 0.000
2      | 7  | Pedro       | 30   | 26-35       | Tucumán     | 1              | 400
| 0.074 | 400.000
...

Monto_TX | Monto_TX_Clip_IQR | Monto_TX_Norm | Monto_IVA | Categoria      | Mes_TX
542.30   | 542.30            | 0.064         | 644.94    | Electrónica    | 2025-01
1200.45  | 1200.45           | 0.128         | 1428.54   | Hogar          | 2025-01
```

5.3 Agregados Generados

Resumen por Ciudad × Categoría (10 filas):

Ciudad	Categoría	Transacciones	Clientes_Únicos	Monto_Total_TX	Monto_Promedio_TX
Buenos Aires	Electrónica	18	4	12540.50	697.25
Buenos Aires	Hogar	12	3	8120.00	676.67
Córdoba	Moda	8	2	4890.00	611.25
La Plata	Deportes	14	3	7250.30	517.88

Pivot: Distribución de Ventas:

text

	Deportes	Electrónica	Hogar	Moda
Buenos Aires	1800.00	12540.50	8120.00	4200.00
Córdoba	950.00	2100.00	1800.00	4890.00
La Plata	3200.00	5600.50	2100.00	1800.00
Rosario	1300.00	800.00	1250.00	2150.00
Salta	2100.50	4200.00	3100.00	1650.00
Tucumán	800.00	1500.00	950.00	1300.00
Mendoza	1150.00	2800.00	1200.00	2400.00
Santa Fe	2300.00	3900.00	2150.00	1800.00
Neuquén	1900.00	4100.00	2200.00	2100.00
B. Blanca	1800.00	3200.00	1800.00	2200.00

5.4 Archivos Exportados

Ubicación: ./output/

- dataset_final.csv** (80 × 14)
 - Dataset limpio, transformado, listo para análisis
 - Formato estándar para importación en herramientas BI
- dataset_final.xlsx** (80 × 14)
 - Mismo contenido que CSV, formato Excel
 - Facilita uso en empresas con ecosistema Excel/Power BI
- reporte_completo.xlsx** (múltiples hojas)
 - Hoja 1:** clientes_limpios (10 clientes + variables derivadas)
 - Hoja 2:** transacciones_limpias (80 transacciones preparadas)
 - Hoja 3:** dataset_final (union completa)
 - Hoja 4:** agregados (10 agregaciones ciudad × categoría)
 - Hoja 5:** pivot_monto_total (matriz ciudad × categoría)
 - Hoja 6:** melt_metricas (forma larga para dashboards)

- **Hoja 7:** tabla_web (demostración de read_html)

5.5 Métricas Clave del Dataset Final

Clientes (n=10):

- Edad: media=31.1, min=25, max=45, sin nulos
- Monto total: media=\$2185, min=\$0, max=\$4100
- Ticket promedio: media=\$582.80
- Distribución geográfica: 10 ciudades (1 cliente cada una)

Transacciones (n=80):

- Monto: media=\$1247.50, mediana=\$945, min=\$0, max=\$2325 (post-clip)
- Distribución temporal: 2025-01 a 2025-04 (4 meses)
- Categorías:
 - Electrónica: 25 TX (31%)
 - Hogar: 20 TX (25%)
 - Deportes: 18 TX (23%)
 - Moda: 17 TX (21%)
- Clientes activos: 9 de 10 (Carlos sin transacciones)

Agregados (n=10):

- Valor medio por grupo (ciudad × categoría): \$1345
 - Grupo más activo: Buenos Aires × Electrónica (\$12,540.50, 18 TX)
 - Grupo menos activo: Tucumán × Moda (\$1,300, 2 TX)
 - Concentración: Top 3 grupos = 39% del volumen total
-

6. Conclusión y Recomendaciones

6.1 Estado Final del Proyecto

✓ Completado satisfactoriamente:

- ✓ Pipeline modular en Python (6 lecciones integradas)
- ✓ Datos limpios: 0 nulos, 0 duplicados
- ✓ Outliers tratados: IQR + Z-score, conservando filas
- ✓ Variables enriquecidas: 6 nuevas columnas derivadas
- ✓ Múltiples formatos de salida: CSV, Excel, hojas temáticas

- ✓ Documentación clara: archivo resumen (este documento)

6.2 Recomendaciones para Próximos Pasos

Análisis Avanzado:

- Correlación edad ↔ ticket promedio (segmentación por demográfica)
- Estacionalidad por mes (peaks de venta)
- Análisis de afinidad (clientes que compran múltiples categorías)

Modelado Predictivo:

- Predicción de churn (clientes que dejarán de comprar)
- Segmentación RFM (Recency, Frequency, Monetary)
- Recomendación de productos (colaborativo)

Escalabilidad:

- Si datos crecen a millones: considerar Dask (paralelización de Pandas)
- Pipeline de automatización: DAG con Apache Airflow
- Data warehouse: migrar a SQL (PostgreSQL, Snowflake)

Monitoreo Continuo:

- Validación de nulos/duplicados en nuevas cargas
 - Alertas si outliers exceden umbral histórico
 - Control de calidad pre-exportación
-

Apéndice: Referencias y Recursos

Librerías Utilizadas

- **NumPy 1.24+**: Operaciones numéricas y generación aleatoria
- **Pandas 2.0+**: Manipulación y análisis de datos
- **Python 3.9+**: Lenguaje base

Documentación Oficial

- [NumPy Docs](#)
- [Pandas Docs](#)

Técnicas Referenciadas

- IQR (Interquartile Range): Estadística robusta para outliers
- Z-score: Normalización y detección de anomalías
- Min-Max Normalization: Escalado a [0, 1]
- GroupBy Aggregation: SQL-like en Python
- Pivot Tables: Cambio de forma (wide ↔ long)